



MAHATMA EDUCATION SOCIETY'S
PILLAI COLLEGE OF ENGINEERING

Department of Electronics and Telecommunication Engineering

Java Programming
PROJECT REPORT

INVENTORY MANAGEMENT SYSTEM
USING JAVA SWING

Submitted By

Name: CHORAGE ADITYA UTTAM

Roll Number: 510

Class: T. Y. EXTC A

Academic Year: 2026-27

Abstract

Inventory management is the process of recording, monitoring and controlling the stock of items available in an organization. Manual record keeping is slow, error-prone and makes it difficult to search products or to detect items that are running out of stock.

This project presents an **Inventory Management System** developed as a desktop application using **Java** and the **Java Swing** graphical user interface toolkit. The system stores product information such as product ID, name, category, price and quantity in an **ArrayList** of **Product** objects and displays it in a **JTable**. The application supports adding, updating, deleting and searching products, identification of low-stock products (quantity less than 5), and a dashboard showing the total number of products, total quantity, number of low-stock products and the total inventory value.

Input validation and exception handling are implemented to prevent invalid data entry. The project demonstrates important Java concepts such as classes and objects, encapsulation, collections, event handling and GUI development. The current version stores data only in memory; persistent storage using a database, user authentication and report generation are identified as future enhancements.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Project Overview	1
1.3 Objectives	1
1.4 Scope	2
2 Problem Statement	2
2.1 Problems with Manual Inventory Management	2
2.2 Difficulty in Searching Products	2
2.3 Maintaining Accurate Quantity	2
2.4 Identifying Low-Stock Products	2
2.5 Updating and Deleting Records	2
2.6 Need for a User-Friendly GUI	2
2.7 Proposed Solution	2
3 Proposed System	3
3.1 System Overview	3
3.2 System Objectives	3
3.3 Functional Requirements	3
3.4 Non-Functional Requirements	3
3.5 System Scope	4
4 Technology Review	4
4.1 Java	4
4.2 Object-Oriented Programming	4
4.3 ArrayList	4
4.4 Java Swing	4
4.5 AWT	5
4.6 Exception Handling	5
4.7 Event Handling	5
4.8 Technology Summary	5
5 System Design and Architecture	6
5.1 Working Principle	6
5.2 System Architecture	6
5.3 Product Class Architecture	7
5.4 Main.java	7
5.5 InventoryGUI.java	7
5.6 module-info.java	8
5.7 Project Structure	8

6	Technologies Used	8
7	Implementation	8
7.1	Product Class	8
7.2	Constructor	9
7.3	Getter and Setter Methods	9
7.4	ArrayList Implementation	9
7.5	Sample Data	9
7.6	Product Table	9
7.7	Adding Products	9
7.8	Updating Products	10
7.9	Deleting Products	10
7.10	Searching Products	10
7.11	Low-Stock Detection	10
7.12	Inventory Value Calculation	11
7.13	Input Validation	11
7.14	Exception Handling	11
7.15	GUI Event Handling	11
7.16	Application Entry Point	11
8	GUI Description	12
8.1	Application Dashboard	12
8.2	Product Table	12
8.3	Add Product	12
8.4	Update Product	13
8.5	Delete Product	14
8.6	Search Product	14
8.7	Low Stock	14
8.8	Dashboard Calculations	15
9	Testing and Results	15
9.1	Testing Methodology	15
9.2	Test Cases	16
9.3	Sample Results	16
10	Challenges, Limitations and Future Scope	17
10.1	Challenges Faced	17
10.2	Current Limitations	17
10.3	Future Scope	17
11	Conclusion	18

List of Figures

5.1	Working flowchart of the system	6
5.2	System architecture	7
5.3	Class diagram of the Product class	7
8.1	Application dashboard	12
8.2	Add product – input dialog	13
8.3	Product added successfully message	13
8.4	Inventory after adding a new product	13
8.5	Update product – input dialog	13
8.6	Delete product – confirmation dialog	14
8.7	Search dialog	14
8.8	Search results for the text “motor”	14
8.9	Low-stock products	15

List of Tables

3.1	Functional requirements	3
4.1	Technology summary	5
6.1	Technologies used	8
7.1	Button actions	11
9.1	Test cases	16
9.2	Sample inventory data	16
9.3	Inventory value calculation	16

1. Introduction

1.1 Background

Inventory refers to the goods, materials and components that an organization keeps in stock for use or sale. Inventory management is the process of tracking these items so that the right quantity is available at the right time. Accurate inventory tracking helps to avoid shortages, prevents unnecessary over-stocking, reduces losses and supports better purchasing decisions.

In small shops, laboratories and workshops, inventory is often maintained manually in registers or spreadsheets. Such methods are time consuming, difficult to search and prone to human error. A software application can store the records in an organized form, perform calculations automatically and present the information clearly.

Java was selected for this project because it is a widely used, platform-independent, object-oriented language. Java Swing, which is part of the Java standard library, allows the development of desktop graphical user interfaces without any external dependency.

1.2 Project Overview

The Inventory Management System is a desktop-based Java application developed using Java Swing. It allows the user to manage product information including product ID, name, category, price and quantity. The application provides a dashboard, a product table and options to add, edit, delete and search products and to view low-stock items.

1.3 Objectives

The main objectives of the project are:

- To develop a simple inventory management application.
- To maintain product information systematically.
- To implement Create, Read, Update and Delete (CRUD) operations.
- To provide product search functionality.
- To identify low-stock products.
- To calculate the total inventory value.
- To demonstrate Object-Oriented Programming concepts.
- To develop a graphical user interface using Java Swing.
- To implement input validation and exception handling.

1.4 Scope

The scope of the project includes the design, development and testing of a single-user desktop application that manages product records in memory. The system can add, update, delete and search products, detect low-stock items and display summary information.

The current system does not provide permanent storage, user login, multi-user access or network connectivity. These are discussed as future enhancements in Chapter 10.

2. Problem Statement

2.1 Problems with Manual Inventory Management

Manual inventory records maintained on paper are difficult to keep up to date. Entries may be missed, duplicated or written incorrectly, and calculating totals requires repeated manual effort.

2.2 Difficulty in Searching Products

Finding a particular product in a long register requires scanning many entries. This wastes time and increases the chance of overlooking the required item.

2.3 Maintaining Accurate Quantity

The quantity of every product changes frequently as items are issued or purchased. Without a systematic method, the recorded quantity may differ from the actual stock.

2.4 Identifying Low-Stock Products

Products that are about to run out must be identified early so that they can be re-ordered. In manual systems, low-stock items are often noticed only after they are completely finished.

2.5 Updating and Deleting Records

Correcting a price or quantity, or removing an obsolete product, is untidy in a handwritten record and may lead to overwriting or inconsistency.

2.6 Need for a User-Friendly GUI

Users who are not programmers require a simple, visual interface with buttons, tables and dialog boxes rather than text commands.

2.7 Proposed Solution

The proposed Inventory Management System solves these problems by storing each product as a Java object, displaying all records in a table, validating every input, searching by ID or name, automatically flagging products with quantity less than 5 as low stock, and

calculating the total inventory value on a dashboard. A Swing-based graphical interface makes these operations easy to perform.

3. Proposed System

3.1 System Overview

The proposed system is a standalone Java Swing application. On start-up it loads a set of sample products, displays a dashboard with summary cards and shows all products in a table. The user performs operations using buttons, and every operation immediately refreshes the table and dashboard.

3.2 System Objectives

- Provide quick access to product information.
- Reduce data-entry errors through validation.
- Highlight products that need re-ordering.
- Present inventory summary information at a glance.

3.3 Functional Requirements

The functions provided by the system are listed in Table 3.1.

Table 3.1: Functional requirements

Function	Description
Add Product	Adds a new product to the inventory.
View Product	Displays all products in a table.
Update Product	Modifies the details of a selected product.
Delete Product	Removes a selected product.
Search	Searches products by ID or name.
Low Stock	Displays products with quantity less than 5.
Dashboard	Shows the inventory summary.

3.4 Non-Functional Requirements

- **Usability:** simple and clear graphical interface.
- **Reliability:** invalid input is handled without terminating the program.
- **Performance:** operations on the in-memory list respond instantly for small inventories.
- **Maintainability:** code is divided into separate classes with clear responsibilities.
- **Simplicity:** no external libraries or database are required.

3.5 System Scope

The system manages one inventory for one user on one computer. Data exists only while the program is running.

4. Technology Review

4.1 Java

Java is a high-level, object-oriented programming language. Programs are compiled into bytecode that runs on the Java Virtual Machine (JVM), making them platform independent. Java provides a rich standard library, automatic memory management and strong error checking, which makes it suitable for desktop applications.

4.2 Object-Oriented Programming

Object-Oriented Programming (OOP) organizes software around objects. The main concepts used in this project are:

- **Class:** a blueprint describing data and behavior (for example, `Product`).
- **Object:** an instance of a class (for example, the product “Arduino Uno”).
- **Constructor:** a special method that initializes an object when it is created.
- **Encapsulation:** keeping data private and providing access through methods.
- **Methods:** functions that define the behavior of a class.
- **Getters and setters:** methods used to read and modify private attributes.

4.3 ArrayList

`ArrayList<Product>` is a resizable array from the Java Collections Framework. It was chosen because it grows automatically, allows easy addition and removal of elements, supports iteration with the enhanced `for` loop and keeps the elements in insertion order.

4.4 Java Swing

Swing is Java’s graphical user interface toolkit. The components used are:

- `JFrame` – the main application window.
- `JPanel` – containers used to group components.
- `JButton` – clickable buttons for operations.
- `JLabel` – text labels.
- `JTable` – tabular display of products.

- `JScrollPane` – adds scrolling to the table.
- `JOptionPane` – dialog boxes for input and messages.

4.5 AWT

The Abstract Window Toolkit (AWT) supplies supporting classes used with Swing: `BorderLayout` and `GridLayout` for arranging components, and `Color`, `Font` and `Dimension` for styling and sizing.

4.6 Exception Handling

Exception handling allows a program to respond to run-time errors without crashing. In this project, a `try-catch` block catches `NumberFormatException`, which occurs when the user enters non-numeric text in the price or quantity field.

```
try {
    int quantity = Integer.parseInt(quantityText);
}
catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this,
        "Please enter a valid quantity.");
}
```

4.7 Event Handling

Swing uses the event-driven model. When the user clicks a button, an `ActionEvent` is generated and the registered listener is executed. Lambda expressions provide a short way of writing such listeners.

4.8 Technology Summary

Table 4.1: Technology summary

Technology	Purpose
Java	Core programming language
OOP	Structuring the program into classes and objects
<code>ArrayList</code>	Storing product objects
Swing	Building the graphical interface
AWT	Layouts, colors and fonts
Exception handling	Handling invalid input

5. System Design and Architecture

5.1 Working Principle

The complete working of the system is shown in Figure 5.1. After start-up, sample data is loaded and the dashboard is displayed. The user selects an operation, the input is validated, the `ArrayList` is updated, the `JTable` is refreshed and the dashboard is recalculated. If the input is invalid, an error message is shown and control returns to the user.

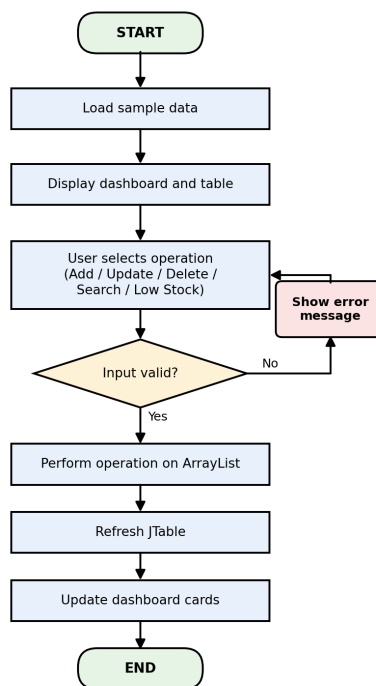


Figure 5.1: Working flowchart of the system

5.2 System Architecture

The architecture of the system is shown in Figure 5.2. The user interacts with the Swing interface; the `InventoryGUI` class handles events and calls the product management operations, which work on `Product` objects stored in the `ArrayList`.

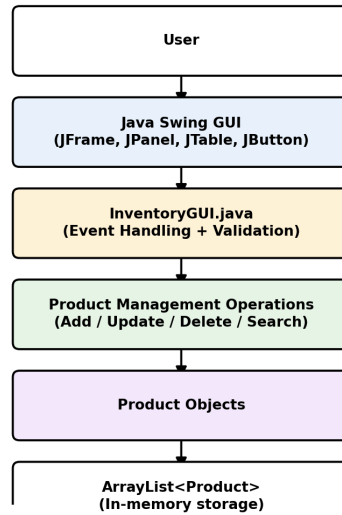


Figure 5.2: System architecture

5.3 Product Class Architecture

The `Product` class models a single inventory item. Its structure is shown in Figure 5.3. All attributes are private and are accessed through public getter and setter methods.

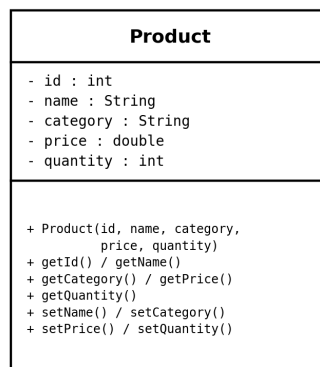


Figure 5.3: Class diagram of the `Product` class

5.4 Main.java

`Main.java` is the entry point of the application. It creates the `InventoryGUI` window on the Swing event-dispatch thread.

5.5 InventoryGUI.java

`InventoryGUI.java` contains the graphical interface and the application logic: the sidebar, dashboard cards, product table, button handlers, validation and the add, update, delete, search and low-stock operations.

5.6 module-info.java

The project uses the Java module system. The declaration requires `java.desktop`; makes the Swing and AWT classes available to the application.

5.7 Project Structure

```
InventoryManagementSystem
|
+-- src
|   +-- main
|       |   +-- Main.java
|       |   +-- Product.java
|       |   +-- InventoryGUI.java
|       |
|       +-- module-info.java
```

6. Technologies Used

The technologies and tools used in the project are listed in Table 6.1.

Table 6.1: Technologies used

Technology	Purpose
Java	Programming language
Swing	Graphical user interface
AWT	Layout and styling
ArrayList	Temporary in-memory storage
JTable	Displaying the inventory
JOptionPane	User input and messages
Exception handling	Error handling
Eclipse IDE	Development environment

7. Implementation

7.1 Product Class

The `Product` class stores the data of one inventory item.

```
private int id;
private String name;
private String category;
private double price;
private int quantity;
```

The variables are declared `private` so that they cannot be changed directly from outside the class. This is the principle of encapsulation: data is protected and accessed only through methods.

7.2 Constructor

```
public Product(int id, String name, String category,  
               double price, int quantity)
```

The constructor receives the product ID, name, category, price and quantity and assigns them to the attributes of the new object.

7.3 Getter and Setter Methods

Getter methods (`getId()`, `getName()`, `getCategory()`, `getPrice()`, `getQuantity()`) return the stored values. Setter methods (`setName()`, `setCategory()`, `setPrice()`, `setQuantity()`) modify them. No setter is provided for the ID so that the identity of a product cannot be changed after creation.

7.4 ArrayList Implementation

```
private ArrayList<Product> products = new ArrayList<>();
```

All products are stored in this list while the program is running.

7.5 Sample Data

```
private void loadSampleData() {  
    products.add(new Product(101, "Arduino Uno", "Electronics", 550, 15));  
    products.add(new Product(102, "HC-SR04 Sensor", "Sensors", 120, 8));  
    products.add(new Product(103, "Jumper Wires", "Accessories", 80, 25));  
    products.add(new Product(104, "Servo Motor", "Motors", 180, 3));  
    products.add(new Product(105, "L293D Motor Driver", "Electronics", 150, 4));  
}
```

7.6 Product Table

The `JTable` and its `DefaultTableModel` display the inventory in rows and columns.

```
String[] columns = {"ID", "Product Name", "Category",  
                   "Price", "Quantity", "Status"};  
tableModel = new DefaultTableModel(columns, 0);  
table = new JTable(tableModel);
```

7.7 Adding Products

When the user adds a product, the entered values are first validated. A new `Product` object is then created and added to the list, and the table is refreshed.

```
products.add(new Product(id, name, category, price, quantity));
refreshTable();
```

Duplicate ID check: before adding, the list is searched to make sure the ID does not already exist.

```
for (Product p : products) {
    if (p.getId() == id) {
        JOptionPane.showMessageDialog(this,
            "Product ID already exists.");
        return;
    }
}
```

7.8 Updating Products

The selected product is modified using its setter methods and the table is refreshed.

```
p.setName(name);
p.setCategory(category);
p.setPrice(price);
p.setQuantity(quantity);
refreshTable();
```

7.9 Deleting Products

The selected Product object is removed from the list and the table is refreshed.

```
products.remove(p);
refreshTable();
```

7.10 Searching Products

The search text is compared with the product ID and, in lower case, with the product name.

```
if (String.valueOf(p.getId()).equals(search)
    || p.getName().toLowerCase().contains(search)) {
    // Display matching product
}
```

7.11 Low-Stock Detection

A product is considered low stock when its quantity is less than 5.

```
if (p.getQuantity() < 5) {
    // Product is low stock
}
```

For the sample data, the Servo Motor (quantity 3) and the L293D Motor Driver (quantity 4) are low-stock products.

7.12 Inventory Value Calculation

The dashboard calculates the value of the inventory as the sum of price times quantity for all products.

```
totalValue += p.getPrice() * p.getQuantity();
```

$$\text{Inventory Value} = \sum (\text{Price} \times \text{Quantity})$$

7.13 Input Validation

The system validates input before changing the inventory: fields must not be empty, the ID and quantity must be integers, the price must be a number, negative values are rejected and duplicate IDs are not allowed.

7.14 Exception Handling

Conversion of text to numbers is placed inside `try-catch` blocks. If the user enters invalid text, a `NumberFormatException` is caught and a message is shown instead of terminating the program (see Section 4.6).

7.15 GUI Event Handling

Each button is connected to an action that runs when it is clicked, as summarized in Table 7.1.

Table 7.1: Button actions

Button	Action performed
Dashboard / Products (sidebar)	Resets the page title and refreshes the table.
Search (sidebar)	Asks for an ID or name and shows matching products.
Low Stock (sidebar)	Shows products with quantity less than 5.
Exit (sidebar)	Asks for confirmation and closes the application.
+ Add Product	Asks for the product details, validates them and adds the product.
Edit	Updates the selected product using setter methods.
Delete	Removes the selected product after confirmation.
Refresh	Displays the complete inventory again.

7.16 Application Entry Point

```
public class Main {
    public static void main(String[] args) {
```

```

        SwingUtilities.invokeLater(() -> {
            new InventoryGUI();
        });
    }
}

```

`SwingUtilities.invokeLater()` ensures that the interface is created on the Swing event-dispatch thread.

8. GUI Description

8.1 Application Dashboard

Figure 8.1 shows the main window of the application. The window is divided into a dark sidebar and a main area.

The sidebar shows the application name and five navigation buttons: **Dashboard**, **Products**, **Search**, **Low Stock** and **Exit**. The main area has the page title and the + **Add Product** button at the top, four dashboard cards below it and the **Product Inventory** table at the bottom with the **Edit**, **Delete** and **Refresh** buttons.

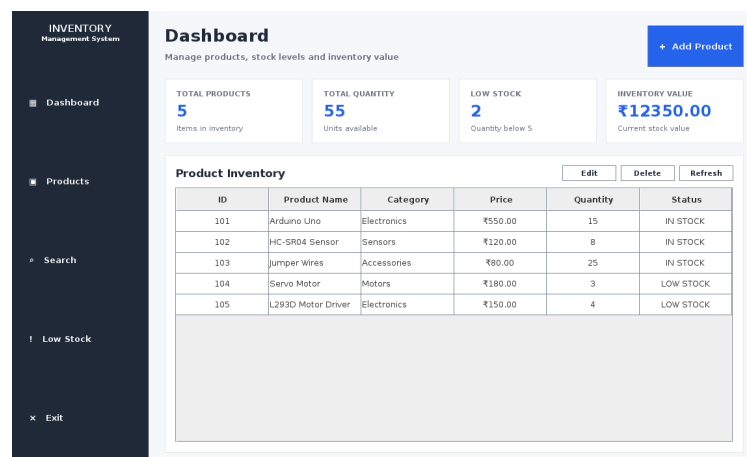


Figure 8.1: Application dashboard

8.2 Product Table

The table has six columns: **ID**, **Product Name**, **Category**, **Price**, **Quantity** and **Status**. The cells of the table cannot be edited directly; changes are made only through the Edit button. The status column shows **IN STOCK** when the quantity is 5 or more and **LOW STOCK** when the quantity is less than 5.

8.3 Add Product

The product is added using the + **Add Product** button. The application asks for the product ID, name, category, price and quantity one after another using input dialog boxes. Figure 8.2 shows the first dialog. If any value is invalid (non-numeric, negative

or duplicate ID) a message is shown and the product is not added. After a successful addition, the message in Figure 8.3 is displayed and the new product appears in the table (Figure 8.4); the dashboard cards are updated automatically.

Figure 8.2: Add product – input dialog

Figure 8.3: Product added successfully message

ID	Product Name	Category	Price	Quantity	Status
101	Arduino Uno	Electronics	₹550.00	15	IN STOCK
102	HC-SR04 Sensor	Sensors	₹120.00	8	IN STOCK
103	Jumper Wires	Accessories	₹80.00	25	IN STOCK
104	Servo Motor	Motors	₹180.00	3	LOW STOCK
105	L293D Motor Driver	Electronics	₹150.00	4	LOW STOCK
106	Raspberry Pi Pico	Microcontrollers	₹350.00	10	IN STOCK

Figure 8.4: Inventory after adding a new product

8.4 Update Product

A product is selected in the table and the **Edit** button is pressed. The application asks for the new name, category, price and quantity, showing the current value in each dialog (Figure 8.5). If no product is selected, the message “Please select a product first” is displayed.

Figure 8.5: Update product – input dialog

8.5 Delete Product

After selecting a product and pressing **Delete**, a confirmation dialog is shown (Figure 8.6). The product is removed only if the user selects **Yes**.

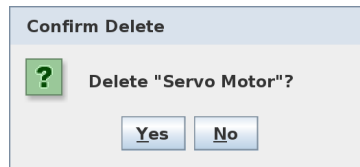


Figure 8.6: Delete product – confirmation dialog

8.6 Search Product

The **Search** button asks for a product ID or part of a product name (Figure 8.7). The matching products are shown in the table and the page title changes to “Search Results” (Figure 8.8). If nothing matches, the message “No product found” is shown and the full table is restored.



Figure 8.7: Search dialog

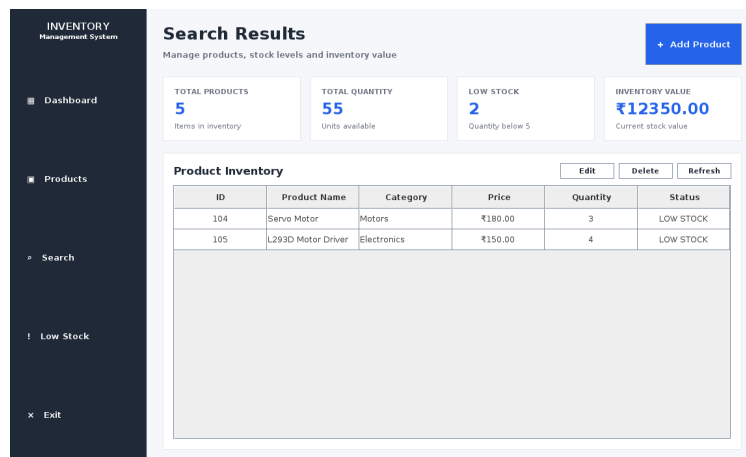


Figure 8.8: Search results for the text “motor”

8.7 Low Stock

The **Low Stock** button shows only the products with quantity less than 5 and changes the page title to “Low Stock Products” (Figure 8.9).

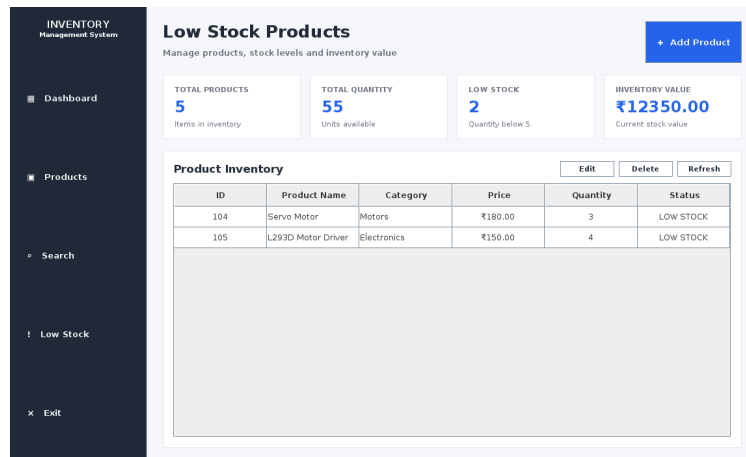


Figure 8.9: Low-stock products

8.8 Dashboard Calculations

The dashboard displays four values, which are recalculated every time the table is refreshed:

- **Total Products:** the number of items in the list.
- **Total Quantity:** the sum of all quantities.
- **Low Stock:** the number of products with quantity less than 5.
- **Inventory Value:** the sum of price $\times$ quantity, displayed with two decimal places.

9. Testing and Results

9.1 Testing Methodology

The application was tested manually (black-box testing). Each function was exercised with valid and invalid inputs and the observed behavior was compared with the expected behavior.

9.2 Test Cases

Table 9.1: Test cases

ID	Test	Expected Result	Result
TC01	Add valid product	Product added	Pass
TC02	Duplicate ID	Warning displayed	Pass
TC03	Invalid price	Error displayed	Pass
TC04	Negative quantity	Rejected	Pass
TC05	Update product	Data updated	Pass
TC06	Delete product	Product removed	Pass
TC07	Search by ID	Product found	Pass
TC08	Search by name	Product found	Pass
TC09	Low stock	Low-stock products displayed	Pass
TC10	Refresh	Full table displayed	Pass

9.3 Sample Results

The sample data loaded at start-up is given in Table 9.2.

Table 9.2: Sample inventory data

ID	Product	Category	Price (Rs.)	Qty
101	Arduino Uno	Electronics	550	15
102	HC-SR04 Sensor	Sensors	120	8
103	Jumper Wires	Accessories	80	25
104	Servo Motor	Motors	180	3
105	L293D Motor Driver	Electronics	150	4

The inventory value is calculated as follows (Table 9.3).

Table 9.3: Inventory value calculation

Product	Price	Qty	Value (Rs.)
Arduino Uno	550	15	8250
HC-SR04 Sensor	120	8	960
Jumper Wires	80	25	2000
Servo Motor	180	3	540
L293D Motor Driver	150	4	600
Total		55	12350

The dashboard therefore shows: Total Products = 5, Total Quantity = 55, Low Stock

= 2 (Servo Motor and L293D Motor Driver) and Inventory Value = Rs. 12350.00 (shown on the dashboard as the rupee amount 12350.00).

10. Challenges, Limitations and Future Scope

10.1 Challenges Faced

- Designing a clear and well-organized GUI.
- Managing the `JTable` and keeping it synchronized with the list.
- Handling invalid user input.
- Managing the `ArrayList` of products.
- Connecting buttons to the correct methods.
- Understanding the many Swing components and layouts.

10.2 Current Limitations

1. **No permanent storage:** data is stored in an `ArrayList`, so it is lost when the program closes.
2. **No login system:** anyone can use the application.
3. **No database:** records are not saved externally.
4. **Single-user desktop application:** it cannot be used by several users over a network.

10.3 Future Scope

The following features are possible future enhancements; they are not part of the current system:

- Database storage using MySQL and JDBC.
- File storage using CSV.
- Login and authentication with user roles.
- Product images.
- Charts for stock and value analysis.
- Sales and purchase management.
- Stock history.
- Invoice generation and PDF reports.

11. Conclusion

An Inventory Management System was successfully developed using Java and Java Swing. The application stores product information as objects in an `ArrayList`, displays it in a `JTable` and supports adding, updating, deleting and searching products, low-stock detection and a dashboard with summary values.

The project demonstrates important Java concepts including classes and objects, encapsulation, constructors, getter and setter methods, collections, event handling, input validation, exception handling and GUI design. Testing with valid and invalid inputs showed that all implemented functions work as expected.

The current version has limitations: data is not stored permanently and there is no user authentication or database. These can be overcome in future by adding a database, login system and report generation, making the project a basis for a more complete inventory solution.

References

- [1] Oracle, “Java Platform, Standard Edition Documentation,” Available at: <https://docs.oracle.com/en/java/>
- [2] Oracle, “The Java Tutorials: Creating a GUI with Swing,” Available at: <https://docs.oracle.com/javase/tutorial/uiswing/>
- [3] Oracle, “The Java Tutorials: Collections,” Available at: <https://docs.oracle.com/javase/tutorial/collections/>
- [4] Eclipse Foundation, “Eclipse IDE,” Available at: <https://www.eclipse.org/>